

EMOJI E CORES NO CONSOLE PYTHON

Lógica de Programação
Escola de Tecnologia UNISENAC

Prof. Fábio Giulian Marques

Porto Alegre, Abril 2025

Sumário

1.Emoji no console Python.....	3
1.1. Usando os códigos Unicode dos emojis	3
1.2. Usando a biblioteca <code>emoji</code> (a maneira mais recomendada):.....	4
1.3. Considerações sobre o ambiente do console:	5
1.4 Resumo:.....	5
2. CORES NO CONSOLE EM PTYHON	6
2.1. Entendendo os Códigos de Escape ANSI:	6
2.2 Cores de Texto (Foreground) Comuns:	6
2.3. Cores de Fundo (Background) Comuns:.....	7
2.4. Usando os Códigos no Python:.....	7
2.5. Considerações Importantes:	8
2.6 Como usar <code>colorama</code> :	9
2.6.1 Instale:	9
2.6.2 Importe e use:	9
2.7 Resumo:.....	9

1.Emoji no console Python

O suporte direto a emojis no console padrão do Python pode ser um pouco... *peculiar*. Isso acontece porque a forma como os caracteres são codificados (o famoso "encoding") varia entre sistemas operacionais e até mesmo entre diferentes configurações de terminal.

Existem algumas maneiras de contornar isso e exibir seus emojis favoritos.

1.1. Usando os códigos Unicode dos emojis

Cada emoji tem um código Unicode único. Podemos usar esses códigos dentro de strings no Python para representá-los. A sintaxe geral é `\Uxxxxxxxx`, onde `xxxxxxxx` é o código Unicode do emoji em hexadecimal (8 dígitos).

- Como descobrir o código Unicode? Existem vários sites e tabelas online que listam os emojis e seus respectivos códigos Unicode. Um bom ponto de partida é a [lista completa de emojis da Unicode Consortium](https://unicode.org/emoji/charts/full-emoji-list.html#1f600).

<https://unicode.org/emoji/charts/full-emoji-list.html#1f600>

Procure pelo emoji desejado e você encontrará o código na coluna "Code". Geralmente, ele aparece com um "U+" seguido por números hexadecimais.

- Exemplo: O código Unicode do emoji de coração vermelho ❤️ é `U+2764`. No Python, você o representaria como `\U00002764` ou `\u2764`. Quando usar **'U' maiúsculo** utilizar 8 dígitos Hexadecimais. Quando possível, ou seja, quando o código tem somente 4 dígitos válidos pode usar **'u' minúsculo**.

Python

```
print('\U00002764') # Saída: ❤️
print('\u2764')    # Saída: ❤️
print('Olá! \U0001F600') # Saída: Olá! 😊
```

Observações importantes sobre códigos Unicode:

- Perceba que adicionamos zeros à esquerda do código original (2764 virou 00002764) para completar os 8 dígitos exigidos pela sintaxe `\U`.
-
- Alguns emojis mais recentes podem ter códigos Unicode maiores, exigindo uma sintaxe ligeiramente diferente (com `\u` para códigos de 4 dígitos ou usando bibliotecas específicas, como veremos adiante).
-

1.2. Usando a biblioteca `emoji` (a maneira mais recomendada):

A biblioteca `emoji` torna o trabalho com emojis no Python muito mais fácil e intuitivo. Ela fornece uma maneira de usar os nomes dos emojis (apelidos) em vez dos códigos Unicode complexos.

- Como instalar: Se você ainda não tem a biblioteca instalada, abra seu terminal ou prompt de comando e execute:

```
Bash
pip install emoji
```

- Como usar:

```
Python
import emoji

print(emoji.emojize(':thumbs_up:'))          # Saída: 👍
print(emoji.emojize('Eu amo Python! :snake:')) # Saída: Eu amo
Python! 🐍
print(emoji.demojize('Olá 🖐️'))              # Saída: Olá :waving_hand:
```

- `emoji.emojize(':nome_do_emoji:')`: Substitui o nome do emoji (entre dois pontos) pelo caractere do emoji correspondente. Você pode encontrar a lista de nomes de emojis suportados na documentação da biblioteca `emoji` ou em diversas fontes online.
- `emoji.demojize('texto com emoji')`: Converte um texto contendo emojis em seus respectivos nomes (útil para armazenar ou transmitir emojis de forma textual).

Vantagens de usar a biblioteca `emoji`:

- Mais fácil de ler e escrever: Usar nomes como `:heart:` é muito mais intuitivo do que lembrar códigos Unicode.
- Maior compatibilidade: A biblioteca lida com as diferenças de codificação de forma mais robusta, aumentando a chance de os emojis serem exibidos corretamente em diferentes consoles.
- Funcionalidades extras: A biblioteca oferece outras funcionalidades úteis para trabalhar com emojis.

Atenção: Algumas tabelas utilizam o nome do emoji separado por espaços em branco, Exemplo: **‘grinning face’**

Para utilizar em Python coloque um **‘_’** entre as palavras, exemplo:

```
print(emoji.emojize(':grinning_face:'))
```

1.3. Considerações sobre o ambiente do console:

Mesmo usando as técnicas acima, a exibição correta dos emojis ainda depende de alguns fatores do seu ambiente:

- Codificação do terminal: O terminal que você está usando precisa estar configurado para uma codificação que suporte emojis, como UTF-8. A maioria dos sistemas modernos já usa UTF-8 por padrão, mas pode haver exceções.
- Fonte do terminal: A fonte usada pelo seu terminal precisa ter glifos (representações visuais) para os emojis que você deseja exibir. Algumas fontes mais básicas podem não ter suporte completo a emojis.

1.4 Resumo:

A maneira mais didática e recomendada de usar emojis no console do Python é através da biblioteca `emoji`. Ela simplifica o processo e oferece maior compatibilidade. No entanto, é sempre bom estar ciente de que a exibição final pode depender das configurações do seu terminal.

Experimente as duas abordagens e veja qual funciona melhor para você e para os seus objetivos! 😊

2. CORES NO CONSOLE EM PYTHON

Cor nas letras do seu console Python pode deixar a saída dos seus programas mais informativa e visualmente agradável. No entanto, o método para fazer isso não é nativo do Python e depende um pouco do sistema operacional e do terminal que você está usando.

A principal forma de adicionar cores é através de códigos de escape ANSI. Esses códigos são sequências especiais de caracteres que o terminal interpreta para aplicar formatação, como cores, negrito, itálico, etc.

Vamos resumir como usar cores:

2.1. Entendendo os Códigos de Escape ANSI:

- Os códigos de escape ANSI geralmente começam com o caractere de escape `\033` (em octal) ou `\x1b` (em hexadecimal), seguido por `[` e um ou mais códigos de formatação separados por `;` e terminando com `m`.
- Para cores de texto (foreground), usamos os códigos na faixa de 30 a 37.
- Para cores de fundo (background), usamos os códigos na faixa de 40 a 47.
- Existem também códigos para estilos, como negrito (1), itálico (3), sublinhado (4), etc.
- Para resetar a cor e a formatação para o padrão, usamos o código 0.

2.2 Cores de Texto (Foreground) Comuns:

Código ANSI	Cor
30	Preto
31	Vermelho
32	Verde
33	Amarelo
34	Azul
35	Magenta
36	Ciano
37	Branco

2.3. Cores de Fundo (Background) Comuns:

Código ANSI	Cor
40	Preto
41	Vermelho
42	Verde
43	Amarelo
44	Azul
45	Magenta
46	Ciano
47	Branco

2.4. Usando os Códigos no Python:

Para exibir texto colorido, você precisa inserir esses códigos de escape dentro da sua string que será impressa com a função `print()`.

Exemplo:

Python

```
print("\033[31mTexto vermelho\033[0m")
print("\033[32mTexto verde\033[0m")
print("\033[34m\033[43mTexto azul com fundo amarelo\033[0m")
print("\033[1m\033[35mTexto magenta em negrito\033[0m")
```

Resultado:

```
Texto vermelho
Texto verde
Texto azul com fundo amarelo
Texto magenta em negrito
```

Explicando o exemplo:

- `\033[`: Inicia a sequência de escape ANSI.
- `31`: Código para a cor vermelha do texto.
- `32`: Código para a cor verde do texto.
- `43`: Código para a cor amarela do fundo.
- `34`: Código para a cor azul do texto (aplicado depois do fundo).
- `1`: Código para negrito.
- `35`: Código para a cor magenta do texto (aplicado depois do negrito).
- `0`: Reseta todas as formatações para o padrão. É muito importante usar o `\033[0m` no final para que as cores não "vazem" para as próximas linhas de saída no console.

Outro exemplo:

```
RESET = "\033[0m"
BOLD = "\033[1m"
CYAN = "\033[96m"
GREEN = "\033[92m"
RED = "\033[91m"
BLUE = "\033[94m"
WHITE = "\033[97m"

print(f"{BOLD}{CYAN}THOR{RESET}  Espécie:{BLUE} Canino {RESET},
{BOLD}{RED} 5 anos{RESET}  Dono:{GREEN} João {RESET}")
```

Resultado

```
THOR Espécie: Canino , 5 anos Dono: João
```

Process finished with exit code 0

2.5. Considerações Importantes:

- Compatibilidade: Os códigos de escape ANSI são amplamente suportados em sistemas Unix/Linux e em muitos terminais no Windows (como o Terminal do Windows e alguns emuladores). No entanto, o console padrão do Windows (cmd.exe) pode ter suporte limitado ou exigir configurações adicionais para funcionar corretamente.
- Bibliotecas de Terceiros: Para uma maneira mais amigável e com maior compatibilidade entre plataformas, existem bibliotecas de terceiros como `colorama`. Essa biblioteca traduz os códigos de escape ANSI para o sistema operacional subjacente, facilitando o uso de cores mesmo no console padrão do Windows.

2.6 Como usar colorama:

2.6.1 Instale:

Bash
`pip install colorama`

2.6.2 Importe e use:

Python

```
from colorama import Fore, Back, Style, init

# Inicialize colorama para Windows
init(autoreset=True)

print(Fore.RED + "Texto vermelho")
print(Fore.GREEN + "Texto verde")
print(Fore.BLUE + Back.YELLOW + "Texto azul com fundo amarelo")
print(Style.BRIGHT + Fore.MAGENTA + "Texto magenta em negrito")
print(Style.RESET_ALL + "Texto normal novamente") # Reseta as cores e estilos
```

A biblioteca `colorama` oferece constantes como `Fore.RED`, `Back.GREEN`, `Style.BRIGHT`, tornando o código mais legível. O `init(autoreset=True)` faz com que as cores sejam resetadas automaticamente após cada `print()`, evitando a necessidade de adicionar `Style.RESET_ALL` manualmente em cada linha.

2.7 Resumo:

- A forma principal de usar cores no console Python é através de códigos de escape ANSI.
- Você pode inserir esses códigos diretamente nas suas strings de `print()`, lembrando de resetar a formatação com `\033[0m` no final.
- Para maior compatibilidade entre plataformas e uma sintaxe mais amigável, a biblioteca de terceiros `colorama` é altamente recomendada.

Experimente os exemplos e veja qual método funciona melhor no seu ambiente! 😊